# Evolving Chart-QA Inference for a 2B VLM

An AlphaEvolve-style system, manual baselines, and an honest holdout

Ahmadreza Farvardin

October 2026

# The problem

- **Task:** answer CharXiv *descriptive* chart questions (titles, axis labels, tick values, counts, legends, layout) with **Qwen3-VL-2B**, Instruct and Thinking variants.
- **Scoring:** exact match after `strip()`, case-insensitive: answer *format* matters as much as reading the chart.
- **Constraints:** greedy decoding only; only these two models (quantized allowed); speed counts too.
- **Deliverable:** the function `vlm_inference(image_path, question) -> str`, optimized (1) by hand and (2) by an AlphaEvolve-style evolution system, for both models.
- **Data:** 1,000 labelled validation figures $\times$ 4 questions; the task gives a 128-query dev set.

# Approach: three tracks, one clean test

1. **Manual optimization**: one change at a time, measured, committed.
2. **Routing**: number-reading / counting questions to Thinking, the rest to Instruct.
3. **Evolution**: an LLM rewrites the inference code; fitness on a separate split.

| split | queries | role |
|---|---|---|
| dev | 128 | manual design |
| evo | 400 | evolution fitness |
| holdout | 800 | **final test only** |
| reserve | 2,672 | single-use checks |

Disjoint by figure. All times on one RTX 3090; differences under 10 % are noise.

# Baseline: the answers were right, the format was not

- Naive Instruct script: **38.3 %** on dev (49/128), 1.95 s/query; **25.9 %** on holdout.
- Main failure: **verbose answers**. 52 of 128 responses were longer than 60 characters; 3 were correct; they took 82 % of the time. In 34 of 79 wrong answers the gold answer appears verbatim inside.
- **Step 1, answer extraction** (no prompt change, measured exactly by replaying recorded outputs): dev 38.3 → 56.2 % (**+17.9**), fresh reserve 32.0 → 51.7 % (+19.7, 0 broken).
- Naive Thinking script: **0 %** on dev: the 128-token limit stops it mid-reasoning.

# Manual pipeline

| Instruct step | dev | reserve | s/query |
|---|---|---|---|
| baseline | 38.3 | 32.0 | 2.23 |
| 1 extraction | 56.2 | 51.7 | 2.23 |
| 2a more rules | 57.0 | 51.7 | 2.23 |
| 2b answer-only, 64 tokens | 60.2 | 51.2 | **0.41** |
| 2c "Not Applicable" check | **65.6** | **55.2** | 0.44 |

reserve = reserve[:400]; 2a is a negative result.

- 2b: same accuracy, **5.4× faster**.
- 2c: abstention recall 20 → 44 %, precision 100 → 73 %.
- **Thinking**, step 1: answer after `</think>`, 1,024-token budget, forced answer when it runs out: dev 72.7 %.
- Both manual versions then frozen.

# Routing: hypothesis and validation

**Hypothesis**

- Thinking reads numbers better: 24 vs. 16 of 51 (reserve[:128]), 32 vs. 21 of 49 (dev).
- Rule (`route_quant.py`): 7 template phrases → Thinking, else Instruct.
- Routes all 4,000 validation questions as intended (a string-matching check, not generalization).

**Validation on a fresh slice** (reserve 400–800)

- Routing **67.8 %** vs. Instruct alone 58.8 %.
- 49 vs. 13 discordant answers, McNemar $p < 0.001$.
- Cost: 28× slower (10.7 vs. 0.38 s/query).
- Became the pre-registered `best_accuracy` candidate.

# Evolution system design (`alphaevolve/`)

| AlphaEvolve component | this system |
|---|---|
| Program database | islands with ring migration; MAP-Elites grid (time × number-reading accuracy); parent from top-k (70 %) or a random elite — `database.py` |
| Prompt sampler | parent code + per-category scores, failures from evo only, inspirations as diffs, sampled directions, edits already tried — `prompt.py` |
| LLM mutation | SEARCH/REPLACE diffs in the EVOLVE block; Gemini 3.1 Flash-Lite (85 %) + Gemini 3 Flash (15 %) — `llm.py`, `diff.py` |
| Evaluation cascade | static checks → 8 → 64 → 400 evo queries (Thinking 4/32/128), noise-aware promotion — `evaluator.py`, `checks.py` |
| Loop | synchronous generations; atomic checkpoint; push to `evo/<run_id>`; exact resume — `loop.py` |

# Deviations from AlphaEvolve, and additions

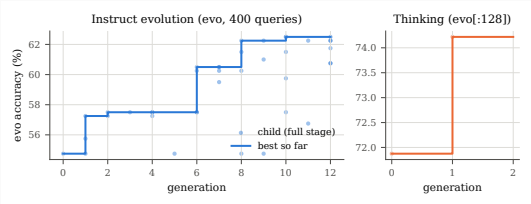
## Deviations (and why)

- Lockstep generations: one GPU; exact resume.
- No meta-prompt evolution: fixed, sampled directions.
- Small scale: 72 + 6 children; a full evaluation costs 2.6 min (Instruct) to $\sim$1 h (Thinking).
- Cascade = growing slices of one benchmark.
- Flash-Lite + Flash instead of Flash + Pro.

## Additions (each after a measured failure)

- **Near-duplicate filter**: reject a child whose edit matches a sibling's (Jaccard $\geq 0.8$) before GPU.
- **Noise-aware selection**: $<$10 % time differences never decide.
- **Static rule checks**: greedy only, allowed model, no gold data.
- **"Already tried" memory** in the prompt.
- **Pause/resume** on API failure; `PAUSE` file.

# Evolution results




**Instruct** (12 gen. × 6, \$0.23)

- evo: 54.75 → 62.50 %; two lineages converge.
- **holdout: 56.0 → 62.0 % (+6.0,** $p = 0.0003$**)**, from number reading (132 → 174 of 326).

**Thinking** (2 gen. × 3, \$0.013)

- edit: reasoning budget 1,024 → 512.
- **holdout**: **−33 % time** (28.6 → 19.3 s/q), **+2.2 points** (64.8 → 67.0 %, $p = 0.01$).

# The routing reversal (headline methodological finding)

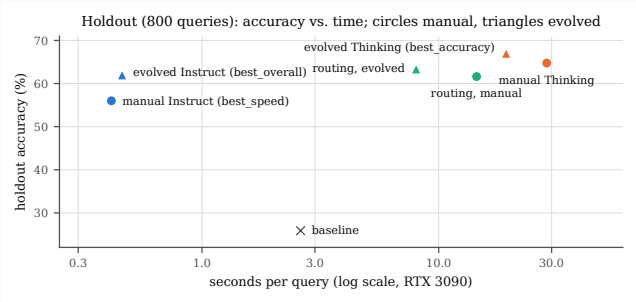
| | dev | holdout |
|---|---|---|
| Instruct alone (manual) | 65.6 | 56.0 |
| routing (manual models) | **74.2** | 61.6 |
| Thinking alone (manual) | 72.7 | **64.8** |

Thinking is also better on OCR-type (+19) and layout (+5) questions, which the small design slice had not shown.

- Routing was validated against **Instruct alone**: it won ($p < 0.001$).
- Against **Thinking alone** on holdout it loses.
- Selection then moved to a *second* fresh slice (reserve 800–1000), not to holdout.

A design validated against the wrong baseline can fail on the right one. We caught this only because holdout had been preserved.

# Final results on holdout (800 queries)




- **best_accuracy**: evolved Thinking, **67.0 %**, 19.3 s/q (chosen on reserve 800–1000).
- **best_overall**: evolved Instruct, **62.0 %**, 0.46 s/q.
- **best_speed**: manual Instruct, **56.0 %**, 0.41 s/q (strict rule).
- Naive baseline: 25.9 %, 2.61 s/q.

# Overfitting audit

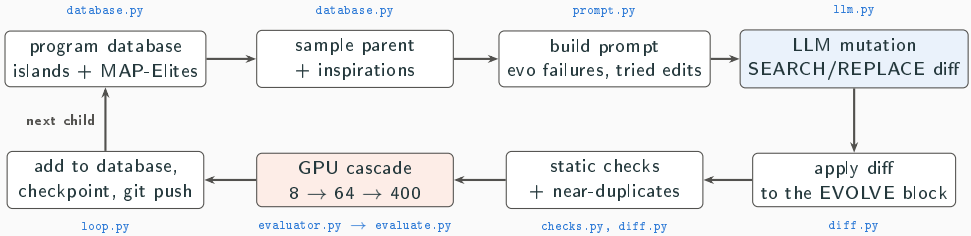
| split | used for | its numbers are |
|---|---|---|
| dev | manual rules, prompts, token limit | fitted (dev−reserve gap grew 4.5 → 10.4 points) |
| reserve[:400] | keep/drop decisions | partly tuned |
| reserve[400:800] | routing validation (once) | honest single use |
| reserve[800:1000] | best_accuracy / best_speed selection (once) | honest single use |
| evo | evolution fitness | training numbers |
| holdout | final comparison only | **the primary evidence** |

- Holdout informed one decision (`best_overall`), where the margin makes the bias negligible.
- The routing phrases come from CharXiv's template source code, not from its answers.

# Repository: which file does what

| README deliverable / file | what it is |
|---|---|
| `manual_instruct.py`, `manual_thinking.py` | hand-optimized pipelines (frozen at step 2c / step 1) |
| `evolve_instruct.py`, `evolve_thinking.py` | the evolution systems: run configuration + CLI over `alphaevolve/` |
| `evolved_instruct.py`, `evolved_thinking.py` | best evolved programs (`g010_c01`, `g001_c00`) |
| `best_accuracy.py` | = evolved Thinking (selected on reserve 800–1000) |
| `best_speed.py` | = manual Instruct |
| `best_overall.py` | = evolved Instruct |
| `evaluate.py` | evaluator: original scoring + CLI, splits, detailed scores JSON |
| `starting_scripts.py`, `route_quant.py` | naive baseline; routing rule (manual design) |
| `reproduce.sh` | one command: setup check, tests, all programs on dev (`-holdout`, `-evolve`) |
| `alphaevolve/` | database, prompt, llm, diff, checks, evaluator, loop, cli |
| `scripts/` | replay, candidate runner, figures, zip builder, LLM check, queue scripts |
| `scores/`, `population/`, `analysis/` | every evaluation, every evolution run, probe data |
| `report/`, `RESULTS.md`, `CONTRACT.md` | the paper; the results log; controller/runner contract |

# How the evolution loop works

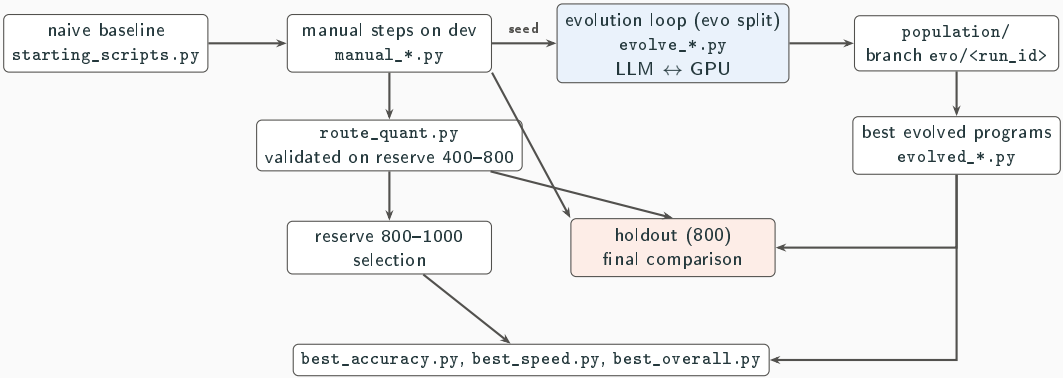



Rejected children (duplicates, invalid edits, rule violations) never reach the GPU. A child passes the 64-query check if it is within a margin of its parent or clearly faster; only full-stage children enter the grid.

# Infrastructure and operations

- **Hardware**: rented RTX 3090 (24 GB), VS Code Remote-SSH; earlier baseline on a Colab T4.
- **Mutation LLM**: Gemini via ArvanCloud's OpenAI-compatible gateway (`api.arvancloudai.ir/v1`): Gemini's API is not available where the machine is hosted. Cost $0.24 for both reported runs.
- **Git**: repo-only deploy key on the GPU; one branch per evolution run.
- **Autonomous runs**: chained tmux queues (evolution → holdout jobs), each job committing its own scores.
- **Robustness**: checkpoint per generation, resume from cache, retry vs. pause on API errors, `PAUSE` file, push timeout.
- **Claude Code**: AI coding assistant for code, experiment runs and documentation; the author made and reviewed the decisions.

# End-to-end workflow




`reproduce.sh` re-runs the evaluation side; `scores/` holds every number shown.

# Lessons and limitations

- **Evolution prefers easy mutations**: it halved the Thinking budget; a measured intervention (stop before the 4th "Wait") cut ∼60 % of reasoning tokens with accuracy kept or raised, and was suggested first.
- **Diversity failure**: 4 of 6 Thinking children were byte-identical despite the "already tried" list.
- **Cascade**: net 10 % GPU saving on Instruct (filters +28 min, early stops +24, small-stage overhead −36): cheap full evaluations and a 75 % pass rate make it break even; it pays off for Thinking.
- **Negative result**: more extraction rules did not help (dev +1, reserve +0).
- **Method**: validate against the strongest baseline, and keep one split untouched.
- **Limits**: small runs (72 + 6 children); one GPU; speed work (fast processor, compiled decoding, vLLM) deferred.

# Questions?

**Repository (private):** `github.com/ahmadrezafarvardin/evolving-vlm-inference`

`bash reproduce.sh` **(dev, ∼2–3 h)**

`bash reproduce.sh -holdout` **(+∼14 h)**

`python evolve_instruct.py` **(needs** `ARVAN_API_KEY`**)**